

Директива component_variants

два режима, четыре фазы — один компонент за один canvas-вызов всегда

A Два режима

single или bulk — выбираешь сам

> single

component_variants Button

- один конкретный компонент
- точечная доработка
- для демо и обучения

> bulk

component_variants

- агент идёт по ds/components.md
- на каждом неполном — бриф
- ты управляешь скопом по списку

...le-прогонов под капотом

B Четыре фазы (одинаковы для обоих режимов)

в bulk просто повторяются на каждом компоненте

1 Read

> ds/components.md + get_metadata

Имя, Node ID, текущая матрица

Сверка с Figma — на расхождение пометит

2 Brief

> что добавляем

variants / states / sizes — три вопроса

Любую категорию можно пропустить

3 md-апдейт

> Semantic под состояния

surface-action-*-hover → accent-700

Новые токены — alias к Primitive, не hex

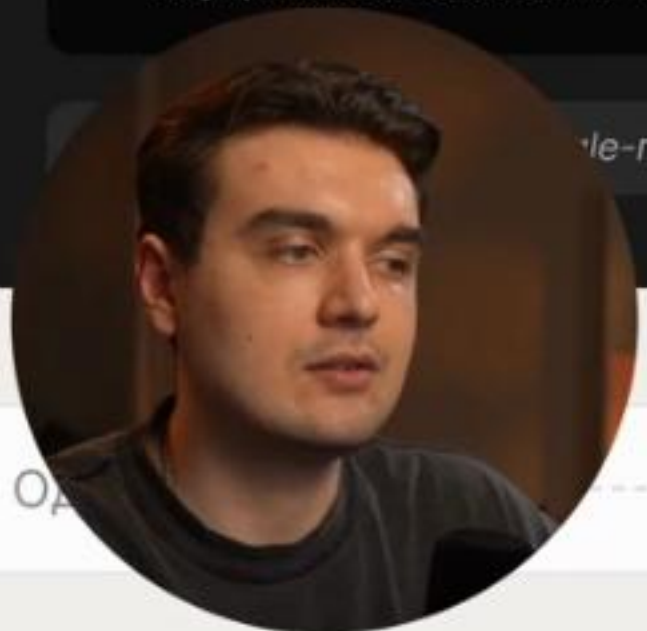
4 Canvas

> один use_figma на компонент

Component Set, имена через слэш

Button/Primary/Hover/Md

Сквозной инвариант: все fills/strokes — через setBoundVariable. Прибитый hex запрещён



Од

— один canvas-вызов. Write-tools плывут на длинных пакетах

Что от директивы ожидать

размер матрицы — главный предиктор стабильности



Стабильно — до 12-15 ячеек

Типовая стабильная матрица:

> пример

3 variants × 4 состояния = 12 ячеек

За один прогон собирает:

- Component Set с матрицей
- все fills через Semantic
- имена через слэш, Figma собрала автоматически
- 5-7 минут точечной доводки руками

> что за этим скрывается

Несколько файлов под состояния
...ds/foundation.md



Сбоит — от 25+ ячеек

Где write-tools плывут:

> пример

5 variants × 5 состояний = 25+ ячеек

Что плывёт:

- часть привязок отваливается до hex
- Auto Layout разваливается на ячейках
- focus ring через прибитый цвет, не Variable
- loading с анимацией — каркас да, анимация нет

> обход

Дробим на 2-3 прогона: сначала states,
потом variants, потом sizes



— условие, чтобы Figma собрала Component Set автоматически

Sizes как ещё одна variant-ось

не отдельный механизм — то же brief, тот же Component Set, третье измерение матрицы



Что меняется на размере

всё через *Primitive*, не прибитые числа

> padding

space-1 (sm) / space-2 (md) / space-4 (lg)

> font-size

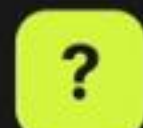
size-sm / size-base / size-lg

> height (если нужно фиксировать)

height-sm / height-md / height-lg

Через ту же директиву, тот же *Component Set* —
просто третье измерение матрицы

`component_variants Button «добавь sizes sm, md, lg»`



Где размеры нужны, где нет

не плодим впрок



Button — sm / md / lg

основные действия + компактные



IconButton — sm / md

тулбары + основные действия



Card — обычно один размер

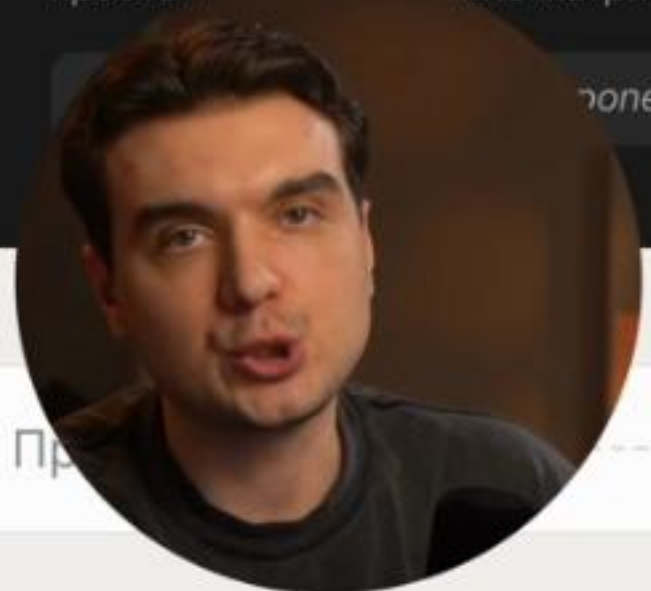


Tooltip / Badge — один размер достаточно

> матрица $3 \times 4 \times 3 = 36$ ячеек

write-tools на пределе

если плывёт — дробь: sizes к одному variant сначала



Пр

— добавляем размеры там, где они реально используются на экранах

Когда руками, когда через MCP

директива — для пакетной постановки матрицы. Микро-правки — Figma напрямую

M Через MCP *component_variants*

- ✓ Первая постановка матрицы
- ✓ 3-4 variants × 4-5 состояний
- ✓ Новая variant-ось — sizes
- ✓ Bulk по всему UI-киту

Почему MCP:

массив однотипной работы плюс синхрон
с *ds/foundation.md* и *ds/components.md*

на длинной матрице

H Руками в Figma *3 секунды быстрее, чем промт*

- ~ +1 variant в готовый Set
- ~ Точечная правка одной ячейки
- ~ Корректировка отступа в Hover
- ~ Подгонка размера спиннера

Почему руками:

микро-действие, промт стоит больше
времени, чем само действие в Figma

не тратить токены на 3-секундную правку

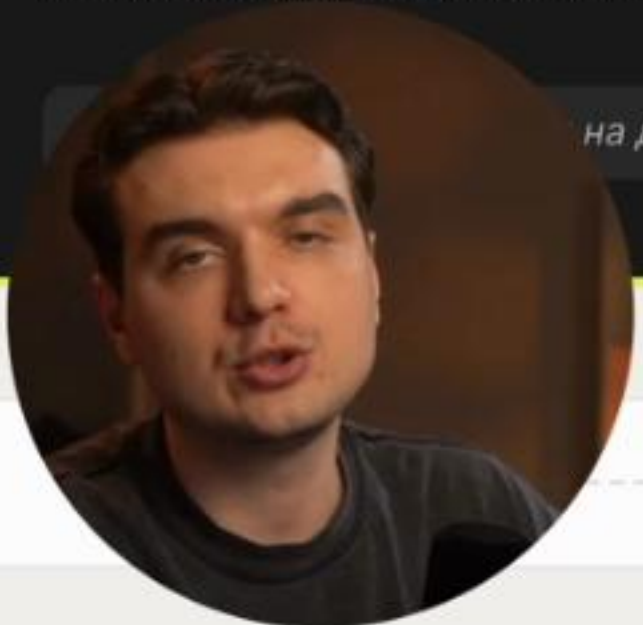
× Не через MCP вообще *это не AI-задача*

- ✗ Анимация loading shimmer
- ✗ Smart Animate переходы
- ✗ Interactive components
- ✗ Вариативный hover с переходом

Это инструменты Figma:

Smart Animate / interactive components
ставятся вручную, не через write-tools

директива оставляет каркас под анимацию



— массивная работа через MCP, микро-правки руками